

Software Bug Prediction using Machine Learning on JM1 Dataset

Nowrin Muhaimin Shailee
Department of Computer Science
American International University-Bangladesh (AIUB)
Dhaka, Bangladesh
nowrin.ms56@gmail.com

Tanvir Ahmed
Department of Computer Science
American International University-Bangladesh (AIUB)
Dhaka, Bangladesh
tanvir6626@gmail.com

Api Alam
Department of Computer Science
American International University-Bangladesh (AIUB)
Dhaka, Bangladesh
apialam008@gmail.com

Rifat Al Mamun Rudro
Department of Computer Science
American International University-Bangladesh (AIUB)
Dhaka, Bangladesh
rifat.rudro138964@gmail.com

Kamruddin Nur
Department of Computer Science
American International University-Bangladesh (AIUB)
Dhaka, Bangladesh
kamruddin@aiub.edu

Abstract—Ensuring the quality of software systems is essential for effective and efficient usage in complex software development procedures. A vital component of the whole procedure is the early identification and forecasting of potential defects or issues in software components. Using the NASA-curated JM1 dataset, this study examines the prediction accuracy of various machine learning techniques, such as Naïve Bayes, Decision Trees, Random Forest, Support Vector Machine, Logistic Regression, Artificial Neural Networks, and K-Nearest Neighbors for detecting defects in software. The research highlights the significance of early defect discovery in software development using machine learning. The experimental results showcase the Random Forest model as the most effective, with high accuracy (81%), precision, recall, and low Root-Mean-Square Error. Furthermore, this study includes a comparative analysis with prior works, offering valuable insights into the performance and efficacy of the proposed methodology. The study concludes by suggesting future research directions.

Index Terms—Software bug prediction, Faults prediction, Prediction model, Machine learning

I. INTRODUCTION

Ensuring the quality and accuracy of software systems is crucial for successful outcomes in the complex software development process. Early detection and prediction of possible defects or issues in software modules is a crucial component in the whole process. Early bug detection is essential for improving software quality and customer satisfaction and lowering the overall cost of software maintenance. The particular JM1 dataset shows proof of how software engineering data is

evolving. This NASA-curated dataset contains actual events from the JM1 software, a real-time predictive ground system that makes predictions using simulations. The difficulties of identifying defective modules get more complex as software systems get more complex. Determining these issues is crucial to developing workable solutions to improve software system quality. In this environment, machine learning methods have proven to be effective instruments for anticipating and reducing defects in software. Software engineering is only one of the many fields where machine learning, specifically supervised learning algorithms, has shown great potential. These algorithms are well-suited for software that predicts module problems because they can identify patterns and relationships within huge datasets. Because of this, the JM1 dataset will be used in this study's investigation of the efficacy of various machine learning algorithms.

Through highlighting particular features of the JM1 dataset, this study seeks to provide significant additional information on the field of software quality prediction. The difficulties in predicting defective modules, the complexities of the JM1 dataset, and the critical role machine learning perform in resolving these difficulties will all be covered in the following sections. In Section II, the related work is discussed. Section III covers the datasets using machine learning algorithms and the evaluation process. The experimental results in Section IV, and the conclusions and future work are presented in Section V.

II. RELATED WORK

In recent years, the application of machine learning (ML) algorithms for software bug and defect prediction has gained significant attention. This section reviews several key research papers contributing valuable insights to this growing field. The application of Naïve Bayes, Decision Tree, and Artificial Neural Networks for software bug prediction was investigated in the paper [1]. The research, based on three real testing and debugging datasets, highlights Decision Trees as the most effective algorithm—Naïve Bayes, Decision Tree, and Artificial Neural Networks outperforming linear autoregression (AR) and the known power model (POWM) models in terms of RMSE. Precision measures revealed that all ML algorithms were effective in bug prediction, with average precision values surpassing 97%. The DT classifier demonstrated the highest recall value, while ANNs and NB achieved average recall values of 99% and 96%, respectively. The F-measure comparison illustrated that DT outperformed ANNs and NB. Delphine Immaculate et al. [2] proposed a systematic methodology for bug prediction using decision tree, logistic regression, random forest classifiers, and naive Bayes. Their study, based on datasets from 15 Java and Python projects on GitHub, demonstrates the superiority of the random forest model with a 97% accuracy rate. The paper emphasized the importance of rigorous data preprocessing but acknowledged limitations such as the absence of exploration into artificial neural networks and the need for generalizability across diverse projects.

Sharma et al. [3] presents a comprehensive overview of Machine Learning Techniques including Neural Networks, Support Vector Machine (SVM), Decision Trees, Ensemble Learning algorithm, Evolutionary algorithms etc. Tanaka et al. [4] contributed by evaluating the effectiveness of *auto-sklearn*, an automated ML library, in predicting defects in software modules across multiple releases. The findings revealed comparable performance to the random forest model, showcasing the potential of automated approaches in defect prediction. The impact of differences in defect causes across releases and the significance of data amount in automated machine learning efficacy were both emphasized in the study. X. Zhao et al. [5] introduced the Importance Descending Order Exclusion (IDOE) method for software fault prediction, comparing the performance of six ML algorithms. The study identified Random Forests as optimal for larger datasets and LibSVM for smaller ones. While acknowledging variability in algorithm effectiveness across datasets, the paper contributed valuable insights into attribute selection methods and classifier suitability for specific dataset sizes.

Aydin and R. Samli [6] systematically evaluated five ML classification algorithms across NASA defect prediction datasets. The study emphasized Random Forest's dominance with consistently high success rates, providing clear numerical performance metrics. While suggesting future research directions, the paper overlooked explicit mention of limitations, potentially requiring further investigation into dataset imbalance or overfitting. P and Kambli [7] proposed a comprehensive

bug prediction model employing various ML algorithms, with a focus on feature selection and cross-validation. Based on the PROMISE JM1 dataset, the study revealed that Artificial Neural Networks (ANN) outperformed other algorithms with an impressive 82.77% accuracy. Despite acknowledging dataset specificity, the paper's contribution lay in providing a robust methodology for bug prediction, considering essential aspects like feature selection and model validation. Prabha and Shivakumar [9] introduced a hybrid model combining Principal Component Analysis (PCA) for feature reduction with various ML algorithms. The findings highlighted Support Vector Classifier (SVC) achieving the highest accuracy of 98.70% in predicting defects across different datasets. The study recognized the necessity for more datasets for extra validation and stressed the influence of feature selection strategies on classifier accuracy.

Manjula et al. [10] studied various classification techniques for software defect prediction, including supervised, unsupervised, and semi-supervised methods. The paper underscored the importance of early defect detection, utilizing quantitative software metrics. In comparison, offering insights into the strengths and weaknesses of different techniques. Khalid et al. [11] contributed to software defect prediction by employing machine learning techniques, feature selection, and K-means clustering. Their study achieved notable results on the CM1 dataset, with SVM and optimized SVM models outperforming others, reaching accuracy rates of 99% and 99.80%, respectively. Applying Particle Swarm Optimization (PSO) and ensemble techniques further refined the models, although limitations included the constrained dataset size. Malhotra [13] categorized ML techniques, and identified commonly used metrics and datasets. Notably, Random Forest models consistently outperformed others, emphasizing the need for more comparisons between ML and Logistic Regression techniques.

III. METHODOLOGY

In this section, dataset descriptions are presented along with the names and labels of their respective attributes. Then, we selected machine learning algorithms for software bug prediction. Based on the selected machine learning algorithms, we evaluate the machine learning model performance metrics on F1-score, precision, recall, accuracy, and RMSE values as shown in Fig. 1.

A. Dataset

The NASA Metrics Data Program makes available the JM1 dataset, which is specifically designed for software fault prediction [8]. The dataset, which consists of 10,885 instances and 22 attributes, includes static code metrics such as branch count, lines of code metrics, and McCabe's and Halstead's measurements [5], [6]. JM1, which is developed in "C" functions as a real-time predictive ground system that makes predictions based on simulations. Using this dataset, researchers have investigated how different learners perform in defect prediction, highlighting the importance of false alarm rates, detection probability, and the influence of effort on

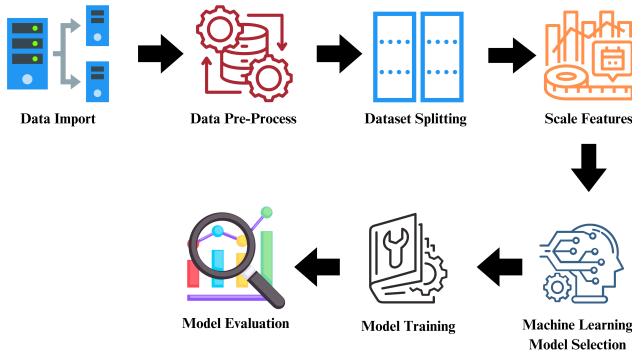


Fig. 1. Methodology Workflow

predictive metrics [6]. The attribute sequence of JM1 is shown in Table I. In this work, to assess the model's performances, the dataset was split into training and testing sets, allocating 20% of the samples for testing and the remaining 80% were used for training.

TABLE I
ATTRIBUTES OF JM1 DATASET [8]

Attribute	Labels
loc	McCabe's line count of code
v(g)	McCabe "cyclomatic complexity"
ev(g)	McCabe "essential complexity"
iv(g)	McCabe "design complexity"
n	Halstead total operators + operands
v	Halstead "volume"
l	Halstead "program length"
d	Halstead "difficulty"
i	Halstead "intelligence"
e	Halstead "effort"
b	Halstead
t	Halstead's time estimator
lOCode	Halstead's line count
lOComment	Halstead's count of lines of comments
lOBlank	Halstead's count of blank lines
lOCodeAndComment	Numeric
uniq_Op	Unique operators
uniq_Opnd	Unique operands
total_Op	Total operators
total_Opnd	Total operands
branchCount	Percentage of the flow graph
defects	{false, true} - Module has/has not reported defects

B. Used Machine Learning Algorithms

By uncovering hidden patterns in past software data, the methods of machine learning can prevent software failures early in the software development life cycle (SDLC) [21]. Since it is dependent upon the features of the dataset. Selecting the best approach to utilize for fault prediction is challenging [20]. This study aims to perform a thorough analysis of six well-known supervised machine learning algorithms: Naïve Bayes, Decision Trees, Random Forest, Support Vector Machine, Logistic Regression, Artificial Neural Networks, and K-Nearest Neighbors. This study aims to evaluate these ML algorithms' predictive power and performance accuracy

with a focus on software bug prediction. The study offers a thorough comparison analysis that illustrates the advantages and disadvantages of the chosen ML algorithms.

Naïve Bayes (NB): As a probabilistic classifier, Naïve Bayes is distinguished by its simplicity and effectiveness. It functions under the independence assumption between characteristics and is based on the Bayes theorem. NB stands for a collection of algorithms that are based on the same principle: that a feature's existence or absence is unrelated to other features [1] [18].

Decision Tree (DT): Decision trees are robust models that are frequently applied in many domains, such as pattern recognition and machine learning. They take the form of a sequence of tests that compare a numerical property at each node to a threshold value, creating a hierarchical structure [22].

Artificial Neural Network (ANN): ANNs operate as massively parallel systems modeled after biological brain networks. Comprising interconnected artificial neurons, ANNs compute weighted sums of inputs, generating outputs that become inputs to other neurons. This process continues until one or more outputs are produced. In the context of bug prediction, ANNs learn intricate patterns from labeled data [7], [15]. For particular problems, different ANN architectures will produce different outcomes [23].

Random Forest: During training, the Random Forest ensemble learning technique creates several decision trees. It operates as a collection of tree-structured classifiers, providing robust predictions. By randomly permuting (shuffling) a given variable, random forest calculates the relevance of the variable [25]. The ensemble nature of Random Forest helps improve predictive accuracy and generalization [2] [5].

Support Vector Machine (SVM): Often used for binary classification applications, SVM is a supervised machine learning model. It excels when dealing with datasets with limited data, drawing a decision boundary between classes to achieve maximum margins [11].

K-Nearest Neighbors (KNN): KNN is a neighbor-dependent classification technique. To define a KNN classifier, a distance metric is used to measure the closeness of vectors, representing transformed open-source code based on criteria such as design complexity. KNN is suitable for identifying patterns in datasets [16].

Logistic Regression: The relationship between independent variables and the dependent variable is modeled using logistic regression. Logistic Regression can be extended to handle multiclass variables through multinomial logistic regression [17].

C. Evaluation Metrics

For data preprocessing, the label encoding is applied to the target variable. Furthermore, categorical features are encoded using the one-hot method. Next, the features are removed from the target variable and the feature set is then standardized by performing feature scaling.

The study predicts software defects using a variety of machine-learning techniques. Naïve Bayes, Decision Tree, Ar-

tificial Neural Network (ANN), K-Nearest Neighbors (KNN), Random Forest, Logistic Regression, and Support Vector Machine (SVM) are some of these models. Predictions are produced for the test set once each model has been trained on the training dataset. In this work, a set of well-known metrics based on the generated confusion matrices was used to evaluate the performance of ML systems in software bug prediction. The following are the assessment measures and confusion matrix that were used:

1) **Confusion Matrix:** The effectiveness of the machine learning models is assessed using a confusion matrix. This is the most popular method for summarizing a classification algorithm's performance [24]. This matrix provides a detailed examination of True Positives (TP), False Positives (FP), True Negatives (TN), and False Negatives (FN). Numerous assessment measures are computed using the confusion matrix as a foundation [1].

2) **Accuracy:** A classification model's overall correctness is determined by its accuracy. The percentage of correctly identified cases—both True Positives and True Negatives—among all the instances is computed in Equation 1 [1], [7].

$$ACC = \frac{TP + TN}{TP + TN + FP + FN} \quad (1)$$

3) **Precision:** Precision, sometimes referred to as Positive Predictive Value, assesses how well a model predicts the future. True Positives, or accurately expected positive cases, are divided by the total number of expected positive instances shown in Equation 2 (True Positives + False Positives) to determine this ratio [1], [19].

$$Precision = \frac{TP}{TP + FP} \quad (2)$$

4) **Recall:** Recall, known as True Positive Rate, measures how well a model can recognize positive examples depicted in Equation 3. It is the ratio of all correctly predicted positive cases (True Positives) to all actual positive cases (True Positives + False Negatives) [1], [7].

$$Recall = \frac{TP}{TP + FN} \quad (3)$$

5) **F-measure:** The harmonic mean of precision and recall is known as the F-measure. Where Equation 4 offers a single metric for model comparison, striking a compromise between precision and recall. When there is an unequal distribution of classes, the F-measure might be helpful, and it is important to consider both false positives and false negatives [1], [7].

$$F\text{-measure} = \frac{2 \cdot Recall \cdot Precision}{Recall + Precision} \quad (4)$$

6) **Root-Mean-Square Error (RMSE):** Regression tasks frequently use RMSE to calculate the average magnitude of errors between predicted and actual values in Equation 5. Larger mistakes are punished more severely than lesser ones. Regression issues where the size of errors matters and the actual numerical value must be predicted are appropriate for RMSE [1].

$$RMSE = \sqrt{\frac{\sum_{i=1}^n (X_i - XP_i)^2}{n}} \quad (5)$$

IV. EXPERIMENTAL RESULTS

A. Machine Learning Model Performance

The six machine learning techniques in the software bug prediction problem were assessed in this study using Python machine learning. Table II displays all the performance on training and testing sets. With an accuracy of 81%, the Random Forest model outperformed the Logistic Regression model, which came in second place with an accuracy of 80% in the testing set. These models outperformed Naïve Bayes, Artificial Neural Network, Decision Tree, SVM, and K-Nearest Neighbors.

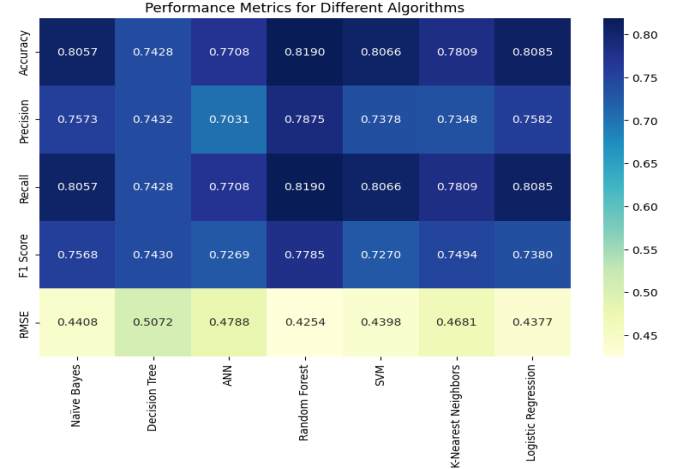


Fig. 2. Performance Metrics for Different Algorithms on Testing Data.

After that, in terms of precision, the Random Forest model performed the best with a precision of 79%, indicating that it had a high accuracy in predicting positive instances. The Logistic Regression model also performed well with a precision of 75%. The next evaluation matrix is recall measures the proportion of actual positive instances that the models correctly identified in Fig. 2. In terms of recall, the Random Forest model performed the best with a recall of 81%, indicating that it effectively identified a high percentage of actual positive instances.

To compare the machine learning models we also used F-measure. Random Forest has the highest F1 score. Therefore, based on the F1 scores, Random Forest appears to perform the best in achieving a balance between precision and recall, followed closely by Logistic Regression. Finally, in evaluating machine learning algorithms using RMSE, Random Forest stands out as the top performer with the lowest value of 0.4254, indicating superior accuracy in predicting outcomes. Logistic Regression also performs well with an RMSE of 0.4377. Other models exhibit slightly higher RMSE values ranging from 0.4408 to 0.5072, suggesting a relatively higher average deviation between predicted and actual values.

B. Discussion

Considering the performance of Random Forest, Logistic Regression, SVM, and Naïve Bayes a comparative analysis

TABLE II
MODEL PERFORMANCE ON TRAINING AND TESTING SETS

Model	Testing Results					Training Results				
	Prec.	Rec.	F1	Acc.	RMSE	Prec.	Rec.	F1	Acc.	RMSE
Naïve Bayes	0.757	0.806	0.757	0.805	0.441	0.757	0.805	0.757	0.805	0.442
Decision Tree	0.743	0.743	0.743	0.743	0.507	0.989	0.989	0.989	0.989	0.103
ANN	0.703	0.771	0.727	0.771	0.479	0.714	0.777	0.734	0.777	0.472
Random Forest	0.788	0.819	0.778	0.819	0.425	0.989	0.989	0.989	0.989	0.103
SVM	0.738	0.807	0.727	0.807	0.440	0.784	0.809	0.730	0.809	0.437
k-NN	0.735	0.781	0.749	0.781	0.468	0.821	0.839	0.818	0.839	0.401
Logistic Reg.	0.758	0.808	0.738	0.808	0.438	0.766	0.809	0.742	0.809	0.437

TABLE III
COMPARISON OF OUR AND PREVIOUS RESEARCH ACCURACY RESULTS

Method	Accuracy Achieved (%)	Previous Accuracy (%)
Random Forest	82.00	77.96 [26]
Logistic Regression	81.00	80.94 [27]
SVM	80.70	80.66 [27]
Naïve Bayes	80.50	80.30 [26]

with other studies on the JM1 dataset showcased in Table III, favorable accuracy results.

In the study [26], the authors employed various machine learning software packages, namely WEKA, SAS, See5, and ROCKY, to analyze the JM1 dataset. The Random Forest model demonstrated an average accuracy of 77.96% and 80.30% for the Naïve Bayes. However, in our work, we demonstrate that employing advanced machine learning techniques on the same dataset results in improved accuracy. In a separate investigation outlined in [27], the authors conducted feature selection on the JM1 dataset, revealing an accuracy of 80.94% for Logistic Regression and 80.66% for SVM. Previously we have found in the section II, where some of the authors in [1] [2] [11], employed different datasets and techniques, our research utilizes a unique dataset and advanced models. Notably, our dataset and models consistently demonstrate superior accuracy, underscoring the effectiveness of our approach compared to prior methodologies

In the context of software bug prediction, Random Forest performs well on the given dataset for several reasons, including its ensemble nature, ability to handle various data types, feature importance analysis, and robustness to overfitting and outliers.

V. CONCLUSION AND FUTURE WORK

The study explored the software bug prediction using the JM1 dataset and machine learning algorithms. The effectiveness of these approaches was proved by the results, which emphasized the Random Forest model, which showed the best accuracy (81%), precision (78.75%), recall (81%), F-measure, and lowest RMSE (0.4254). Where Logistic Regression performed admirably. Future research may investigate advanced feature engineering methods to evaluate their effect on model efficacy. Furthermore, adding contextual data and domain-specific knowledge into models may further improve the overall prediction accuracy.

REFERENCES

- [1] Hammouri, A., Hammad, M., Alnabhan, M., & Alsarayrah, F. (2018, January). Software Bug Prediction using Machine Learning Approach.

- International Journal of Advanced Computer Science and Applications, 9.
- [2] S. Delphine Immaculate, M. Farida Begam and M. Floramary, "Software Bug Prediction Using Supervised Machine Learning Algorithms," 2019 International Conference on Data Science and Communication (IconDSC), Bangalore, India, 2019, pp. 1-7.
- [3] D. Sharma, P. Chandra, "Software Fault Prediction Using Machine-Learning Techniques," in Smart Computing and Informatics, Guru Gobind Singh Indraprastha University, 2018, pp. 541-549.
- [4] K. Tanaka, A. Monden and Z. Yücel, "Prediction of Software Defects Using Automated Machine Learning," 2019 20th IEEE/ACIS International Conference on Software Engineering, Artificial Intelligence, Networking and Parallel/Distributed Computing (SNPD), Toyama, Japan, 2019, pp. 490-494.
- [5] X. Zhao, W. Zhang and N. Ai, "Analysis of Attribute Selection Method Based on IDOE in Software Fault Prediction," 2019 International Conference on Intelligent Computing, Automation and Systems (ICICAS), Chongqing, China, 2019, pp. 741-745, doi: 10.1109/ICIC-CAS48597.2019.00160.
- [6] Z. B. Guven Aydin and R. Samli, "Performance Evaluation of Some Machine Learning Algorithms in NASA Defect Prediction Data Sets," 2020 5th International Conference on Computer Science and Engineering (UBMK), Diyarbakir, Turkey, 2020, pp. 1-3.
- [7] R. P and P. Kambli, "Predicting Bug in a Software using ANN Based Machine Learning Techniques," 2020 IEEE International Conference for Innovation in Technology (INOCON), Bangluru, India, 2020, pp. 1-5.
- [8] PROMISE repository, <http://promise.site.uottawa.ca/SERepository/datasets/jm1.arff> (accessed Dec. 1, 2023).
- [9] C. L. Prabha and N. Shivakumar, "Software Defect Prediction Using Machine Learning Techniques," 2020 4th International Conference on Trends in Electronics and Informatics (ICOEI)(48184), Tirunelveli, India, 2020, pp. 728-733.
- [10] Manjula, C. M., Prasad, Lilly Florence, & Arya, Arti. (2015). A Study on Software Metrics based Software Defect Prediction using Data Mining and Machine Learning Techniques. International Journal of Database Theory and Application, 8(3), 179-190.
- [11] A. Khalid, G. Badshah, N. Ayub, M. Shiraz, and M. Ghouse, "Software Defect Prediction Analysis Using Machine Learning Techniques," Sustainability, vol. 15, pp. 5517, Mar. 21, 2023, DOI: 10.3390/su15065517.
- [12] M. D'Ambros, M. Lanza and R. Robbes, "An extensive comparison of bug prediction approaches," 2010 7th IEEE Working Conference on Mining Software Repositories (MSR 2010), Cape Town, South Africa, 2010, pp. 31-41.
- [13] R. Malhotra, A Systematic Review of Machine Learning Techniques for Software Fault Prediction, Applied Soft Computing Journal (2014).
- [14] Jijo, B. T., & Abdulazeed, A. M. (2021). "Classification Based on Decision Tree Algorithm for Machine Learning." Journal of Applied Science and Technology Trends, 2(1), 20-28.

- [15] I. F. B. Tronto, J. D. S. da Silva, and N. Sant'Anna, "An investigation of artificial neural networks based prediction systems in software project management," *Journal of Systems and Software*, vol. 81, no. 3, pp. 356-367, March 2008.
- [16] M. Hammad, A. Alqaddoumi, H. Al-Obaidy, and K. Almseidein, "Predicting Software Faults Based on K-Nearest Neighbors Classification," *Int. J. Com. Dig. Sys.*, vol. 8, no. 5, Sep. 2019.
- [17] M. A. Ibraigheeth and S. A. Fadzli, "Software project failures prediction using logistic regression modeling," 2020 2nd International Conference on Computer and Information Sciences (ICCIS), Sakaka, Saudi Arabia, 2020, pp. 1-5.
- [18] O. F. Arar and K. Ayan, "A feature-dependent Naive Bayes approach and its application to the software defect prediction problem," *Applied Soft Computing*, vol. 59, pp. 197-209, Oct. 2017.
- [19] M. Efendioglu, A. Sen and Y. Koroglu, "Bug Prediction of SystemC Models Using Machine Learning," in *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 38, no. 3, pp. 419-429, March 2019.
- [20] I. Mehmood et al., "A Novel Approach to Improve Software Defect Prediction Accuracy Using Machine Learning," in *IEEE Access*, vol. 11, pp. 63579-63597, 2023.
- [21] Bilal Khan, Rashid Naseem, Muhammad Arif Shah, Karzan Wakil, Atif Khan, M. Irfan Uddin, Marwan Mahmoud, "Software Defect Prediction for Healthcare Big Data: An Empirical Evaluation of Machine Learning Techniques", *Journal of Healthcare Engineering*, vol. 2021, Article ID 8899263, 16 pages, 2021.
- [22] S. Fletcher and M. Z. Islam, "Decision Tree Classification with Differential Privacy: A Survey," *ACM Computing Surveys*, vol. 52, no. 4, Article 83, pp. 1-33, August 2019.
- [23] M. Saritas and A. Yasar, "Performance Analysis of ANN and Naive Bayes Classification Algorithm for Data Classification," *International Journal of Intelligent Systems and Applications*, vol. 7, 2019.
- [24] N. Ali, D. Neagu, and P. Trundle, "Evaluation of k-nearest neighbour classifier performance for heterogeneous data sets," *SN Applied Sciences*, vol. 1, 2019.
- [25] L. He, R. A. Levine, J. Fan, J. Beemer, and J. Stronach, "Random forest as a predictive analytics alternative to regression in institutional research," *Practical Assessment, Research and Evaluation*, vol. 23, pp. 1-16, 2018.
- [26] L. Guo, Y. Ma, B. Cukic and Harshinder Singh, "Robust prediction of fault-proneness by random forests," 15th International Symposium on Software Reliability Engineering, Saint-Malo, France, 2004, pp. 417-428.
- [27] Azam, Muhammad & Nouman, Muhammad & Gill, Ahsan, "Comparative Analysis of Machine Learning Techniques to Improve Software Defect Prediction", vol 5, pp 41-66, 2022.